ML Sentinel — Presentation Guide

Use this as a 5–7 minute speaking guide with a live demo.

Opening — 20 seconds

ML Sentinel is a mlops & model monitoring platform built to demonstrate both machine-learning reasoning and software-engineering delivery. The demo is local and reproducible, so I can explain every step without depending on a paid service.

Problem — 40 seconds

A trained model can degrade after deployment when the incoming population changes. ML Sentinel demonstrates the model lifecycle: training, versioned quality metrics, online inference, data-drift checks and retraining.

Architecture — 60 seconds

Draw or explain the pipeline from input/data source to model/retrieval logic, then to the FastAPI service and dashboard. Emphasize the separation between model logic and product/API logic.

Live Demo — 2 to 3 minutes

Follow the numbered demo flow below. Pause at the algorithmic output and explain what the model/system is doing rather than clicking quickly through screens.

- Step 1: Show the model quality cards and current version.
- Step 2: Run an online prediction and explain probability vs threshold.
- Step 3: Check a stable batch; PSI remains below 0.2.
- Step 4: Check a shifted batch; drift alert appears.
- Step 5: Retrain and show the model version increment; explain MLflow/Airflow production integration.

Engineering — 45 seconds

Mention validation, tests, Docker, CI and the reason the demo can run locally. Then identify the optional components that would be used in a larger deployment.

Close — 30 seconds

Summarize one technical result, one design trade-off and one next improvement. Invite questions about model choice, metrics, scale or failure handling.

Resume-ready bullets

- Built an end-to-end MLOps service covering data preprocessing, model training, versioned metrics, online inference, retraining and production-style drift monitoring using Population Stability Index (PSI).
- Exposed model-health and prediction APIs with FastAPI, input validation, automated tests, Docker packaging and GitHub Actions CI for repeatable deployment.
- Added optional MLflow experiment-tracking and Airflow retraining integrations, with a modular design for model registry, scheduled retraining and alerting workflows.

Metrics to mention if asked

- ROC-AUC / F1
- Prediction latency
- PSI by feature
- Performance after drift
- Retraining/rollback success

30-second fallback if the live demo fails

> *“The application normally runs locally through FastAPI. I also keep a saved deterministic output and automated tests in the repository. I will show the expected output first, then walk through the code path that produces it and the test that verifies the behavior.”*

Rapid Interview Q&A;

Why PSI?

It is simple, interpretable and commonly used for monitoring tabular feature distribution shifts. It should be combined with model-quality metrics when labels arrive.

When should retraining happen?

Not on drift alone. Use drift severity, business impact, enough new labeled data, performance degradation and approval/rollback policies.

Before the interview

- Run the app once from a clean terminal.
- Run `pytest -q` and confirm all tests pass.
- Know the main API endpoint and one representative request/response.
- Know one limitation and one scale-up plan.
- Do not claim optional technology as trained/deployed unless you have actually run it.